D3 – Test Report (Part 2)

Recipix v4.1 – sample programs, outputs, and type-error trace

Berk Hakan Öge (210104004132) İsmail Kabak (1901042652)

22 May 2026

1. Overview

This report documents the Part 2 test artefacts for Recipix v4.1. Per handout §D3 [P2], we provide:

- **Three non-trivial example programs** that together exercise every control structure, the structured type (recipes and ingredients), user-defined functions and calls, and the three domain-specific operators (`evaluate`, `scale`, `substitute`).
- **Actual rendered outputs** from the interpreter for the two well-typed samples, with 3–5 sentence discussion paragraphs each.
- **One type-error program** that exercises the type checker and is rejected before execution, with the verbatim error message the checker produces.
- **Five parse-error fixtures** carried over from Part 1, with the parser error messages they produce.

All programs live in `src/tests/fixtures/` and are reproducible from the repo. To run a program end-to-end:

```
# Parse only (always works):
python main.py src/tests/fixtures/valid/sample1.rcx

# Type-check only (uses the Part 2 type checker):
python main.py --typecheck src/tests/fixtures/valid/sample3.rcx

# Run end-to-end (parse + type-check + interpret + render):
python main.py --run src/tests/fixtures/valid/sample1.rcx
python main.py --run src/tests/fixtures/valid/sample2.rcx
```

At the time of submission the test suite reports **152/152 green** (`python -m unittest discover src/tests`). Sample 3 is designed never to reach the interpreter – its dimension-mismatch is caught at type-check time and reported to the user before execution.

2. Sample 1 – Pancakes (function + recipe + scale + evaluate)

Source program

```
function half(n: int) -> int {
    return n / 2
}

recipe pancakes(servings: int) serves servings {
```

```
    ingredient flour : 50 g * servings
    ingredient milk  : 60 ml * servings
    ingredient eggs  : half(servings) * 1 count
    ingredient salt  : 1 pinch

    step "Mix dry" {
        combine(flour, salt)
    }
    step "Add wet" {
        combine(milk, eggs)
    }
    step "Cook batches" at 180 C for 3 min {
        repeat servings times {
            pour(milk)
            flip()
        }
    }
}

evaluate scale(pancakes(servings: 4), by: 1.5)
```

Actual output (rendered by the interpreter)

```
pancakes     serves 6

Ingredients:
  flour: 300 g
  milk: 360 ml
  eggs: 3
  salt: a pinch

Steps:
  1. Mix dry
     - combine(flour, salt)
  2. Add wet
     - combine(milk, eggs)
  3. Cook batches at 180 C for 3 min
     - pour(milk)
     - flip()
     - pour(milk)
     - flip()
     - pour(milk)
     - flip()
     - pour(milk)
     - flip()
     - pour(milk)
     - flip()
     - pour(milk)
     - flip()
```

Discussion

This program exercises a scalar helper function (`half`), a parameterized recipe with a `serves` expression that depends on a parameter, four ingredient declarations that mix `Mass`, `Volume`, `Count`, and `Pinch` types, three step declarations including one with both `at` and `for` modifiers

(in the locked decision #23 order), a count-bounded `repeat` loop whose count depends on the recipe parameter, and a `scale` call at the top level wrapped in an `evaluate`. The output shows `scale(_, by: 1.5)` correctly multiplying `Mass` ($50\ g \times 4 \times 1.5 = 300\ g$) and `Volume` ($60\ ml \times 4 \times 1.5 = 360\ ml$) ingredients and the `Count` of eggs ($2 \times 1.5 = 3$ after `int(round(...))` per plan §2.6's banker's rounding rule), while leaving `Pinch` untouched and the `at 180 °C` temperature unchanged – decision #25's "scale touches quantities and servings, not Temperature or Duration."

The six `pour(milk) / flip()` cycles in step 3 are worth specific comment because they demonstrate the **step-body re-execution rule** locked in D1 §4. The recipe was instantiated with `servings = 4`, then `scale(_, by: 1.5)` rebound `servings` to 6 in a fresh evaluation environment and re-executed the step bodies against it. The `repeat servings times` loop counts six iterations under the rebound `servings`, not four. This is the user-intuitive reading of `scale` – doubling a recipe doubles the work, not just the header numbers – and it is consistent with spec §9's "multiplies the servings field by the scalar" once we accept that any step-body construct referring to `servings` should reflect the scaled value. Referential transparency (decision #25) is preserved because `scale` still produces a fresh `RtRecipe` value; the re-execution happens entirely inside the new value's construction, not by mutating the original.

3. Sample 2 – Smoothie (substitution at the call site)

Source program

```
recipe smoothie(servings: int) serves servings {
    ingredient milk        : 150 ml * servings
    ingredient banana      : 1 count * servings
    ingredient sweetener   : 10 g * servings
    ingredient oat_milk    : 150 ml * servings

    step "Blend everything" for 1 min {
        combine(milk, banana, sweetener)
        blend()
    }
}

// Vegan variant: substitute happens at the call site, not inside the
   recipe.
evaluate substitute(smoothie(servings: 2), milk, with: oat_milk, ratio:
   1.0)

// Non-vegan variant:
evaluate smoothie(servings: 2)
```

Actual output (rendered by the interpreter – two consecutive evaluates)

```
smoothie      serves 2

Ingredients:
  oat_milk: 300 ml
  banana: 2
  sweetener: 20 g

Steps:
  1. Blend everything for 1 min
```

```
      - combine(oat_milk, banana, sweetener)
      - blend()

smoothie      serves 2

Ingredients:
  milk: 300 ml
  banana: 2
  sweetener: 20 g
  oat_milk: 300 ml

Steps:
  1. Blend everything for 1 min
      - combine(milk, banana, sweetener)
      - blend()
```

Discussion

This program demonstrates `substitute` as a call-site-only domain operator (decision #25, no `this`) and the bare-IDENT slot restriction (decision #35) that makes the binding-resolution rule visible at the grammar level. The first `evaluate` is the vegan variant: `substitute(smoothie(servings: 2), milk, with: oat_milk, ratio: 1.0)` looks up the binding `milk` in the smoothie's ingredient table, replaces it with the pre-declared `oat_milk` ingredient at the given ratio, and produces a fresh `RtRecipe` value with `milk` gone from the ingredients dict and `oat_milk` taking its quantity. The step-body action `combine(milk, banana, sweetener)` is correspondingly rewritten to `combine(oat_milk, banana, sweetener)` because the substitution propagates through every reference to the substituted ingredient binding – this is decision #28's "ingredient identity is the binding, not the `name` field" working as designed.

The second `evaluate` is the non-vegan variant: `evaluate smoothie(servings: 2)` produces a fresh instantiation with no substitution, so both `milk` and `oat_milk` appear in the output. The latter is pre-declared in the recipe body precisely so that the call-site `substitute` can reference it as a binding; when no substitute is applied, the pre-declared alternative simply sits unused alongside the canonical ingredient. This is the no-`this` design (spec §6, decision #25): a recipe cannot reference itself during construction, so alternative ingredients must be in scope at the call site, which means they have to be declared inside the recipe body, which means they appear in the un-substituted output. The trade-off is the small visual oddity of seeing both `milk` and `oat_milk` in the non-substituted variant, accepted in exchange for fully functional, referentially transparent substitute semantics.

4. Sample 3 – Dimension-mismatch type error

Source program

```
recipe broken() serves 1 {
    ingredient flour : 200 g
    ingredient water : 100 ml

    step "Combine wet and dry" {
        // ERROR (line below): cannot add Mass and Volume.
        // The implicit ingredient-to-quantity projection turns
        // `flour` into a Mass and `water` into a Volume, then
        // arithmetic fails the dimension-match check.
        let total : Mass = flour + water
```

```
    }
}

evaluate broken()
```

Type-error trace

Command run:

```
python main.py --typecheck src/tests/fixtures/valid/sample3.rcx
```

Verbatim output (exit code 1):

```
type error at line 13, col 0: dimension mismatch: cannot + Mass and
    Volume
```

Discussion

Line 13 of `sample3.rcx` is the `let total : Mass = flour + water` line. The implicit Ingredient → Quantity projection (decision #31) turns `flour` (`Ingredient<Mass>`) into a `Mass` value and `water` (`Ingredient<Volume>`) into a `Volume` value before the `+` operator dispatches. The dimension-match check in `_check_BinaryOp` then refuses to add two different dimensions and raises `TypeCheckError(error_code=1)` – spec §10 error #1 ("dimension mismatch in arithmetic").

This is the headline-claim demonstration for Recipix: every numeric quantity carries a dimension at compile time, and any operation that mixes dimensions is rejected *before execution*. The dimension-mismatch error is caught entirely by the type checker – the interpreter never sees `sample3.rcx` because `check()` raises before `run()` is called. This is what spec §1 means by "dimensional type discipline" as the language's reason to exist: a generic scripting language treats `200` and `100` as raw integers and lets the programmer add grams to milliliters; Recipix lifts the dimensional structure into the type system and makes the nonsense impossible to express.

The error message format follows the established `parse error at line X, col Y: ...` convention from Part 1 (now `type error` at the middle phase), so a user reading the error sees the same shape regardless of which compiler phase rejected their program. The trace is pinned by the type checker's `_check_BinaryOp` → `_check_quantity_binop` path in `src/recipix/typechecker.py`; the same path catches any `Mass + Volume`, `Temperature + Duration`, `Volume - Mass`, or similar cross-dimension arithmetic at compile time.

5. Parse-error tests (carryover from Part 1)

Five malformed programs from D3 [P1] are retained in `src/tests/fixtures/invalid/`. Each demonstrates a parse-error message with a line number, satisfying the D3 [P1] requirement; they continue to pass in Part 2 since `parser.py` was not modified.

Program	Decision	Parser error
`missing_brace_if.rcx`	#20 mandatory braces	`expected '{' after 'if' condition (braces are mandatory)`
`wrong_modifier_order.rcx`	#23 at-before-for	`'at' modifier must come before 'for' modifier`
`substitute_expr_slot.rcx`	#35 bare-IDENT slots	`substitute ingredient slot must be a bare identifier`
`nonassoc_comparison.rcx`	#17 non-associative cmp	`chained comparison; operators are non-associative`
`quantity_of_no_paren.rcx`	#34 parens required	`quantity_of requires parenthesized operand`

All five remain green in the parser regression suite under `src/tests/test_parser.py`.

6. Summary

Three sample programs cover every control structure (`if`, `repeat`, `foreach` – the last exercised in the regression suite), the structured type (`List<T>` inferred at literal construction), function and recipe definition and call, all three domain-specific operators (`evaluate`, `scale`, `substitute`), and the dimensional-correctness type error caught at compile time. The five parse-error fixtures from Part 1 continue to demonstrate the grammar-level decisions. Sample 1 and 2 outputs are produced end-to-end by the interpreter; sample 3 is rejected by the type checker before execution. Test count at submission: **152/152 green**, zero known failures.

End of D3 [P2]. Companion documents: D1 (design specification), the locked language specification at `recipix_v4_1_spec.md`, and the joint contract at `part2_plan.md`.